

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Ю. В. Ветрова

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 7

СОЗДАНИЕ СЦЕНАРИЕВ НА JAVASCRIPT

по курсу:

WEB-ТЕХНОЛОГИИ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: с использованием языка JavaScript научиться создавать различные сценарии, выполняемые в web-страницах.

2 Задание

2.1 1-я часть задания.

1. Создайте и подключите к HTML-документу внешний скрипт, выводящий сообщение типа alert с текстом «Здравствуйте! Меня зовут...».

Укажите свои фамилию и имя.

2. Напишите коды объявления функций, используя 3 способа:

- Function Declaration,
- Function Expression,
- Стрелочная функция (arrow function).

Каждая из этих функций должна решать одну и ту же достаточно простую задачу. Вызовите каждую из этих функций по клику на кнопке с соответствующей надписью. Результат выполнения должен быть одинаковым во всех трех случаях.

2.2 2-я часть задания

По 2-ой части задания предусмотрены варианты.

Вариант 6.

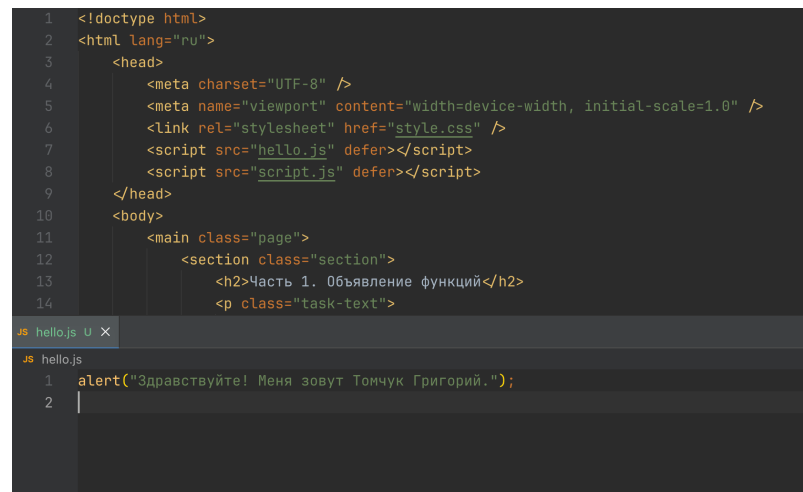
1) Создайте на веб-странице список (например, список обычных дел на воскресенье). Под списком поместите несколько кнопок с надписями, по щелчку на каждой из них в список должен быть добавлен новый элемент, например:

- кнопка «Утренняя пробежка» (или «Зарядка») должна добавить соответствующий элемент в начало списка,
- кнопка «Театр» (или «Каток») должна добавить соответствующий элемент в конец списка,
- кнопка «Выставка» должна добавить соответствующий элемент в середину списка.

2) Создайте веб-страницу с кнопкой, на которой написано «Мои друзья». По клику на кнопке должны поочередно выводиться модальные окна типа "alert" с именами ваших друзей.

3 Ход выполнения

В ходе выполнения лабораторной работы был создан HTML-документ, к которому подключены внешние файлы JavaScript и CSS. Сначала был разработан внешний скрипт hello.js, который при загрузке основной страницы выводит модальное окно с приветствием и указанием фамилии и имени. На рисунке 1 изображено подключение внешнего скрипта к HTML-документу, а на рисунке 2 показан результат его выполнения в браузере.



```
1 <!doctype html>
2 <html lang="ru">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <link rel="stylesheet" href="style.css" />
7     <script src="hello.js" defer></script>
8     <script src="script.js" defer></script>
9   </head>
10  <body>
11    <main class="page">
12      <section class="section">
13        <h2>Часть 1. Объявление функций</h2>
14        <p class="task-text">
```

```
js hello.js
1 alert("Здравствуйте! Меня зовут Томчук Григорий.");
2
```

Рисунок 1 — Скрипт hello.js и его подключение

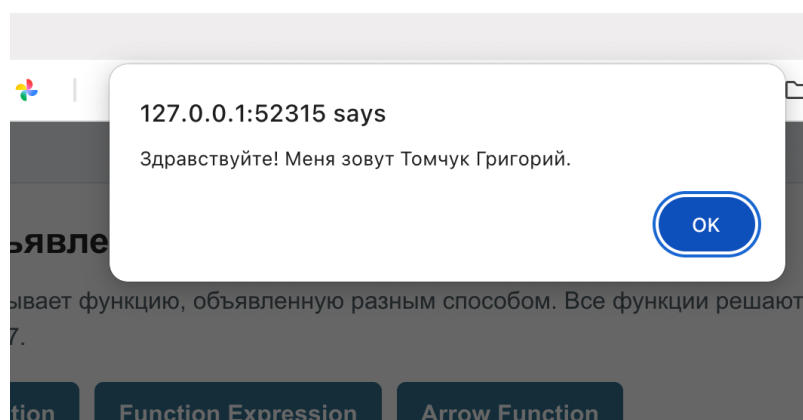


Рисунок 2 — Результат выполнения hello.js

Далее была реализована первая часть задания, связанная с объявлением функций разными способами. В файле script.js были созданы три функции, выполняющие одну и ту же простую задачу: вычисление суммы двух чисел.

Первая функция была объявлена с помощью Function Declaration, вторая с помощью Function Expression, а третья с помощью стрелочной функции. Для каждой функции на странице была создана отдельная кнопка с соответствующей подписью. После нажатия на любую из этих кнопок на странице отображается одинаковый результат вычисления. На рисунке 3 изображен фрагмент кода с тремя способами объявления функций, а на рисунке 4 показан результат работы кнопок на веб-странице.

```
1 function getSumDeclaration() {  
2     return 15 + 27;  
3 }  
4  
5 const getSumExpression = function () {  
6     return 15 + 27;  
7 };  
8  
9 const getSumArrow = () => 15 + 27;  
10  
11 function showFunctionResult(methodName, result) {  
12     const resultElement = document.getElementById("FunctionResult");  
13  
14     if (resultElement) {  
15         resultElement.textContent = `${methodName}: 15 + 27 = ${result}`;  
16     }  
17 }  
18
```

Рисунок 3 — Три способа объявления функции

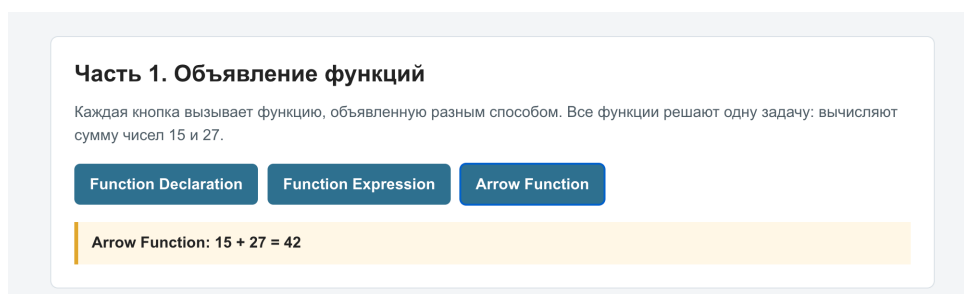


Рисунок 4 — Результат нажатия на кнопку «Arrow Function»

Затем была выполнена первая задача второй части лабораторной работы. На основной веб-странице был создан нумерованный список дел на воскресенье. Под списком были размещены кнопки для добавления новых элементов. При нажатии на кнопку «Утренняя пробежка» новый пункт добавляется в начало списка, при нажатии на кнопку «Театр» пункт добавляется в конец списка, а при нажатии на кнопку «Выставка» элемент вставляется в середину списка. Для этого в JavaScript использовались методы работы с DOM-структурой документа. На рисунке 5 изображен исходный список дел и кнопки управления, а на рисунке 6 показан измененный список

после добавления новых элементов.

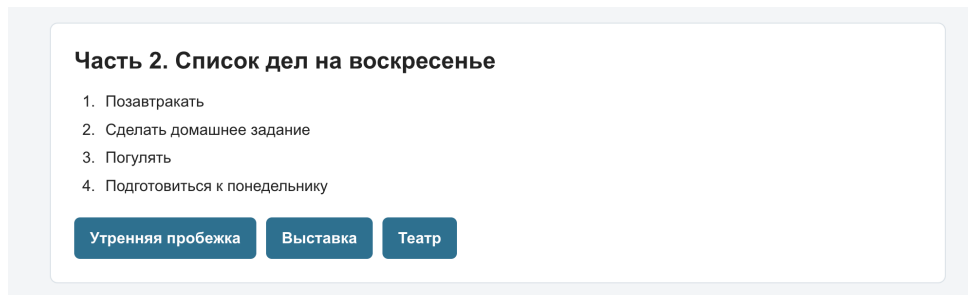


Рисунок 5 — Исходный список дел

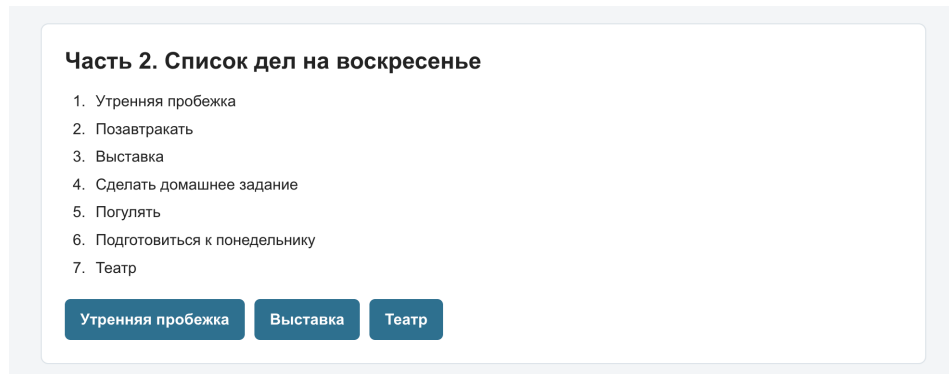


Рисунок 6 — Список дел с добавленными записями с помощью скрипта

После этого была создана отдельная веб-страница `friends.html` для выполнения второй задачи второй части лабораторной работы. На этой странице размещена кнопка «Мои друзья». При нажатии на нее последовательно выводятся модальные окна типа `alert` с именами друзей. Имена были заранее записаны в массив, после чего каждый элемент массива выводится в отдельном модальном окне. На рисунке 7 изображен код функции для вывода имен друзей, а на рисунке 8 показан результат работы кнопки «Мои друзья» в браузере.

```
43 function showFriends() {  
44     const friends = ["Алексей", "Мария", "Иван", "Екатерина"];  
45  
46     friends.forEach((friend) => {  
47         alert(friend);  
48     });  
49 }
```

Рисунок 7 — Функция `showFriends` вывода элементов массива

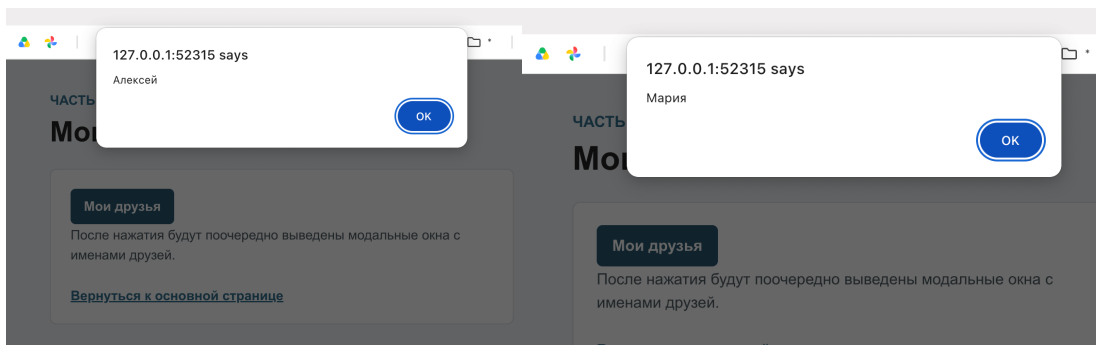


Рисунок 8 — Результат функции showFriends

На завершающем этапе было выполнено оформление страниц с помощью файла style.css. Для элементов страницы были настроены отступы, цвета, размеры шрифтов, внешний вид кнопок и адаптация под небольшую ширину экрана. Также была проверена корректность подключения файлов и синтаксис JavaScript-кода. На рисунке 9 изображено оформление основной и дополнительной страниц после применения стилей.

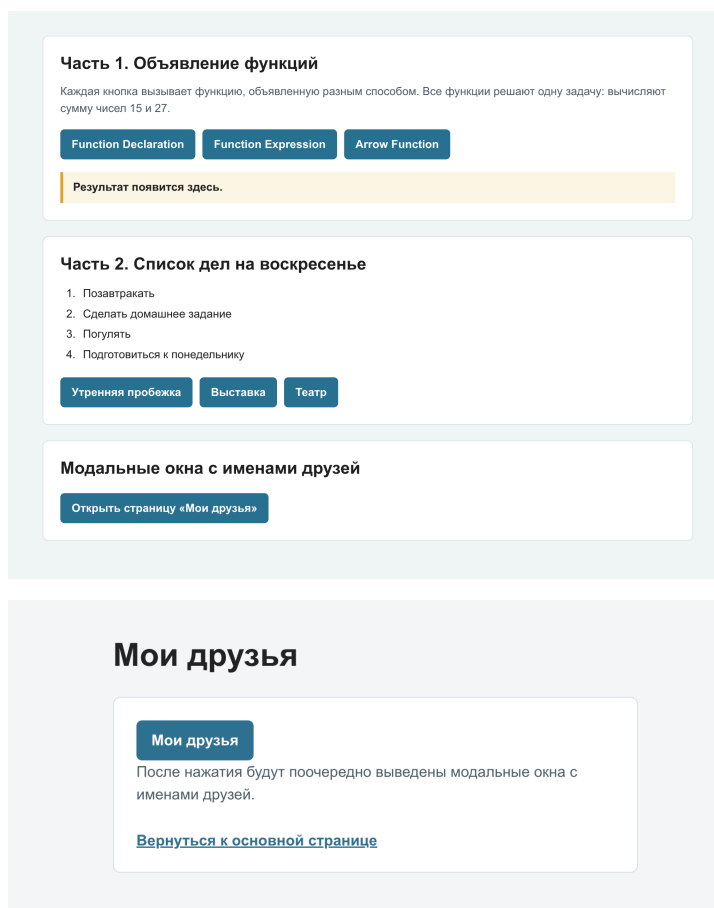


Рисунок 9 — Оформление основной и дополнительной страниц

4 Выводы

В ходе лабораторной работы были получены практические навыки

создания и подключения внешних JavaScript-файлов к HTML-документу. Были рассмотрены три способа объявления функций в JavaScript: Function Declaration, Function Expression и стрелочная функция. Также была изучена обработка событий нажатия на кнопки и вывод результатов работы функций на веб-страницу.

Кроме того, в работе были применены методы взаимодействия с DOM-структурой документа. С их помощью были реализованы добавление элементов в начало, середину и конец списка, а также последовательный вывод модальных окон с помощью alert. В результате была создана веб-страница, демонстрирующая базовые возможности JavaScript для создания интерактивных элементов на сайте.

ПРИЛОЖЕНИЕ

Листинг 1 — index.html

```
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" href="style.css" />
    <script src="hello.js" defer></script>
    <script src="script.js" defer></script>
  </head>
  <body>
    <main class="page">
      <section class="section">
        <h2>Часть 1. Объявление функций</h2>
        <p class="task-text">
          Каждая кнопка вызывает функцию, объявленную разным способом. Все функции
решают
          одну задачу: вычисляют сумму чисел 15 и 27.
        </p>

        <div class="button-row">
          <button type="button" id="declarationBtn">Function Declaration</button>
          <button type="button" id="expressionBtn">Function Expression</button>
          <button type="button" id="arrowBtn">Arrow Function</button>
        </div>

        <p class="result" id="functionResult">Результат появится здесь.</p>
      </section>

      <section class="section">
        <h2>Часть 2. Список дел на воскресенье</h2>

        <ol class="todo-list" id="todoList">
          <li>Позавтракать</li>
          <li>Сделать домашнее задание</li>
          <li>Погулять</li>
          <li>Подготовиться к понедельнику</li>
        </ol>

        <div class="button-row">
          <button type="button" id="addMorningBtn">Утренняя пробежка</button>
          <button type="button" id="addExhibitionBtn">Выставка</button>
          <button type="button" id="addTheatreBtn">Театр</button>
        </div>
      </section>

      <section class="section compact">
        <h2>Модальные окна с именами друзей</h2>
        <a class="link-button" href="friends.html">Открыть страницу «Мои друзья»</a>
      </section>
    </main>
  </body>
</html>
```

Листинг 2 — friends.html

```
<!doctype html>
<html lang="ru">
```



```

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="style.css" />
  <script src="script.js" defer></script>
</head>
<body>
  <main class="page narrow">
    <header class="header">
      <h1>Мои друзья</h1>
    </header>

    <section class="section compact">
      <button type="button" id="friendsBtn">Мои друзья</button>
      <p class="task-text">
        После нажатия будут поочередно выведены модальные окна с именами друзей.
      </p>
      <a class="text-link" href="index.html">Вернуться к основной странице</a>
    </section>
  </main>
</body>
</html>

```

Листинг 3 — script.js

```

function getSumDeclaration() {
  return 15 + 27;
}

const getSumExpression = function () {
  return 15 + 27;
};

const getSumArrow = () => 15 + 27;

function showFunctionResult(methodName, result) {
  const resultElement = document.getElementById("functionResult");

  if (resultElement) {
    resultElement.textContent = `${methodName}: 15 + 27 = ${result}`;
  }
}

function addTodoItem(text, position) {
  const todoList = document.getElementById("todoList");

  if (!todoList) {
    return;
  }

  const item = document.createElement("li");
  item.textContent = text;

  if (position === "start") {
    todoList.prepend(item);
    return;
  }

  if (position === "middle") {
    const middleIndex = Math.floor(todoList.children.length / 2);
    todoList.insertBefore(item, todoList.children[middleIndex]);
    return;
  }
}

```

```

    }

    todoList.append(item);
}

function showFriends() {
    const friends = ["Алексей", "Мария", "Иван", "Екатерина"];

    friends.forEach((friend) => {
        alert(friend);
    });
}

document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("declarationBtn")?.addEventListener("click", () => {
        showFunctionResult("Function Declaration", getSumDeclaration());
    });

    document.getElementById("expressionBtn")?.addEventListener("click", () => {
        showFunctionResult("Function Expression", getSumExpression());
    });

    document.getElementById("arrowBtn")?.addEventListener("click", () => {
        showFunctionResult("Arrow Function", getSumArrow());
    });

    document.getElementById("addMorningBtn")?.addEventListener("click", () => {
        addTodoItem("Утренняя пробежка", "start");
    });

    document.getElementById("addExhibitionBtn")?.addEventListener("click", () => {
        addTodoItem("Выставка", "middle");
    });

    document.getElementById("addTheatreBtn")?.addEventListener("click", () => {
        addTodoItem("Театр", "end");
    });

    document.getElementById("friendsBtn")?.addEventListener("click", showFriends);
});

```

Листинг 4 — style.css

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    background: #f2f5f7;
    min-height: 100vh;
    color: #222;
    font-family: Arial, Helvetica, sans-serif;
}

button,
.link-button {
    transition:
        background 0.2s ease,
        transform 0.2s ease;
    cursor: pointer;
    border: 0;
}

```

```

    border-radius: 6px;
    background: #2f6f8f;
    padding: 12px 16px;
    color: #fff;
    font: inherit;
    font-weight: 700;
}

button:hover,
.link-button:hover {
    transform: translateY(-1px);
    background: #24566f;
}

.page {
    margin: 0 auto;
    padding: 40px 0;
    width: min(920px, calc(100% - 32px));
}

.page.narrow {
    width: min(560px, calc(100% - 32px));
}

.header {
    margin-bottom: 24px;
}

.label {
    margin: 0 0 8px;
    color: #2f6f8f;
    font-weight: 700;
    text-transform: uppercase;
}

h1,
h2,
p {
    margin-top: 0;
}

h1 {
    margin-bottom: 0;
    font-size: 34px;
    line-height: 1.2;
}

h2 {
    margin-bottom: 14px;
    font-size: 24px;
}

.section {
    margin-bottom: 22px;
    border: 1px solid #d9e1e6;
    border-radius: 8px;
    background: #fff;
    padding: 24px;
}

.section.compact {
    margin-bottom: 0;

```

```

}

.task-text {
  color: #4f5d66;
  line-height: 1.5;
}

.button-row {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 18px;
}

.result {
  margin: 18px 0 0;
  border-left: 4px solid #e0a52b;
  background: #fff7e6;
  padding: 12px 14px;
  min-height: 44px;
  font-weight: 700;
}

.todo-list {
  margin: 0;
  padding: 0 0 0 26px;
  line-height: 1.8;
}

.todo-list li {
  padding-left: 6px;
}

.link-button {
  display: inline-block;
  margin-top: 8px;
  text-decoration: none;
}

.text-link {
  display: inline-block;
  margin-top: 12px;
  color: #2f6f8f;
  font-weight: 700;
}

@media (max-width: 560px) {
  .page {
    padding: 24px 0;
  }

  h1 {
    font-size: 28px;
  }

  .section {
    padding: 18px;
  }

  button,
  .link-button {
    width: 100%;
  }
}

```

}
}